

Please watch or skim the videos listed on the Canvas assignment corresponding to this LGA. There is a lot more content than usual in this LGA, since it should be review material for many of you. Feel free to fast-forward though any bits you feel pretty confident about already.

Individual self-test questions

All students should answer this set of questions. You will compare your answers with your group during the next lecture period. Note, we may from time to time include trick questions.

- Roughly how much memory is required to store an object?

The memory used by an object is roughly the sum of the sizes of the member variables in the object. However, most modern CPUs prefer memory values to be aligned along certain boundaries, such as every 4 bytes or every 8 bytes. So if you, for example, create a class that has an int and a char, the size of the object will most likely be 8 rather than 5. The extra space is unused and ignored.

- How would you allocate memory for an object of type `foo` on the stack?

Declare a variable of that type.e.g.,
`foo f`
inside a function.

- How would you allocate memory for an object of type `foo` in the free store?

```
foo* p = new foo;
```

- Last two questions again, assuming you need to pass in an integer argument (0, perhaps).

```
foo f(0);  
foo* p = new foo(0);
```

- How would you allocate memory for an array of `foo` of size 5 on the stack? On the free store?

```
foo foo_array[5];  
foo* foo_array_2 = new foo[5];
```

- Write down function signatures for an overload of the multiplication operator acting on two objects of type `foo` and returning a `foo`, in two ways:

- As a non-member function:

```
foo operator*(const foo& lhs, const foo& rhs);
```
- As a member function of `foo`:

```
foo foo::operator*(const foo& rhs) const;
```

- Write down the function signature for the copy constructor for class `foo`:

```
foo(const foo& arg);
```

- Write down the function signature for the assignment operator overload (as part of the “big 3”) for class `foo`:

```
foo& foo::operator=(const foo& rhs);
```

- The last question again, but assuming `foo` is a template class taking a single type parameter:

```
template <typename T>  
foo<T>& foo<T>::operator=(const foo<T>& rhs);
```

- What are two examples of when the copy constructor is called *implicitly*?

When passing-by-value into a function or returning-by-value from a function.

- What are two ways to invoke the copy constructor *explicitly* for class `foo`?

```
foo f; // (not a copy, just setting up the next lines)  
foo a(f); // a is constructed as copy of f  
foo b = f; // b is constructed as copy of f (provided as initializer)  
foo* p = new foo(f); // dynamically allocated copy of f
```

- What is the difference between copying and assigning an object?

“Copying” occurs both when we make a new object as a copy of another object, and when doing assigning. However, the copy *constructor* is only invoked when making a *new* object that is a copy of an existing object of the same type. Assignment occurs when an *existing* object is overwritten by another object of the same type.

Examples of assignments:

```
foo f, g;  
f = g; // assignment: f already exists, is overwritten by value g  
foo arr[10];  
foo h;  
arr[3] = h; // assignment: foo object in arr[3] already exists
```

- What is the difference between a shallow copy and a deep copy?

A shallow copy simply copies all member variables exactly as they are, including pointers. This will result in unintended shared memory between objects when using the default copy constructor. A deep copy resolves this issue by allocating memory for the new object and copying the contents of the dynamic memory rather than the pointer itself.

Group questions

Distribute the following questions across the members of your group. You will share your solutions (and most importantly the *method* of your solutions) during the next lecture period. Divide up the questions so that each question has at least three solutions from different group members.

1. (Intermediate) What does the following code print to the standard output?

```
class point {
public:
    int x, y;
    point() { x = 0; y = 0; }
    point(int d1, int d2) { x = d1; y = d2; }
};

point operator+(point t, point u) { return point(t.x + u.x, t.y + u.y); }
ostream& operator<<(ostream& out, point p) {
    out << "(" << p.x << ", " << p.y << ")";
    return out;
}

template <typename T>
void print_sum(T* arr, unsigned int len) {
    if (len == 0) return;
    T acc = arr[0];
    for (unsigned int i = 1; i < len; i++) {
        acc = acc + arr[i];
    }
    cout << acc << endl;
}

int main() {
    int iarr[4] = {1, 2, 3, 4};
    print_sum(iarr, 4);

    string sarr[3] = {"blue", "red", "yellow"};
    print_sum(sarr, 3);

    point parr[2];
    parr[0].x = 10;
    parr[0].y = 20;
    parr[1].x = -5;
    parr[1].y = 17;
    print_sum(parr, 2);

    return 0;
}
```

```
10
blueredyellow
(5, 37)
```

2. (Intermediate) Below is `fraction.h` which has the declaration of the `fraction` class. The `fraction` class stores a numerator and denominator in dynamically allocated integers.

`Fraction.h:`

```
class fraction {
private:
    int* num = nullptr;
    int* den = nullptr;
public:
    // constructor
    fraction(int n, int d = 1) {
        num = new int(n);
        den = new int(d);
    }

    // big 3
    fraction(const fraction& orig);
    fraction& operator=(const fraction& orig);
    ~fraction();

    // arithmetic operations
    fraction operator+(const fraction& rhs);
    friend fraction operator*(const fraction& lhs, const fraction& rhs);

    // printing
    friend ostream& operator<<(ostream& out, const fraction& f);
};
```

Complete the fraction class by implementing the “big 3” methods, arithmetic operator overloads (just addition and multiplication, no reduction required), and `ostream` output operator. (Note we have deliberately set this up so you must implement one arithmetic operation as a method of `fraction`, and the other as a free function.) Divide the work up among your group members.

Below is a some test code for your fraction class:

```
int main() {
    fraction a(5, 8);
    fraction b(4, 9);
    fraction c = a;
    cout << (a * b) << endl; // prints 20/72
    cout << (a + b) << endl; // prints 77/72
    cout << (b * c) << endl; // prints 20/72
    a = c = b;
    cout << (a * b) << endl; // prints 16/81
    return 0;
}
```

```
// Since num and den are dynamically allocated, we must write this
// copy constructor to dynamically allocate, so we get our own copy
// instead of sharing with orig
fraction::fraction(const fraction& orig) {
    num = new int(*orig.num);
    den = new int(*orig.den);
}

// Assignment operator; like the copy constructor, but we usually must
// also clean up our old memory. In this case, we didn't need to do that
// since we could simply overwrite the contents of the memory we already had.
fraction& fraction::operator=(const fraction& orig) {
    if (this == &orig) return *this;
    *num = *orig.num;
    *den = *orig.den;
    return *this;
}

// Deallocate to avoid memory leaks
fraction::~fraction() {
    delete num;
    delete den;
    // pedantic --- gaurantees a crash if the object is
    // misused in the future
    num = den = nullptr;
}

// member function (method) operator overload
fraction fraction::operator+(const fraction& rhs) {
    int n = *num * *rhs.den + *den * *rhs.num;
    int d = *den * *rhs.den;
    return fraction(n, d);
}

// non-member function (free function) operator overload
fraction operator*(const fraction& lhs, const fraction& rhs) {
    return fraction(
        *lhs.num * *rhs.num,
        *lhs.den * *rhs.den
    );
}

ostream& operator<<(ostream& out, const fraction& f) {
    return out << *f.num << "/" << *f.den;
}
```

main Test Output:

```
20/72
77/72
20/72
16/81
```

3. (Basic) Implement the template function `print_five()`, which takes in a single value and prints the value 5 times (using `cout`). Write and test code that uses the template successfully with at least 3 different types.

```
template <typename T>
void print_five(T val) {
    for (int i = 0; i < 5; i++) {
        cout << val << endl;
    }
}
```

4. (Basic) Write code to demonstrate what happens when you use the above template with an argument for which there is no defined print behavior.

```
vector<int> vec = {1, 2, 3};
print_five(vec);
```

On my machine, this prints out literally pages of error message, which is really frustrating to look at. The actually useful part of the error message is right at the top:

```
test.cpp: In instantiation of 'void print_five(T) [with T = std::vector<int>]':
test.cpp:26:19:   required from here
test.cpp:20:18: error: no match for 'operator<<' (operand types are
  'std::ostream' {aka 'std::basic_ostream<char>'} and 'std::vector<int>')
 20 |             cout << val << endl;
    |             ~~~~~~
```

The error message tells us that, on line 20 of file `test.cpp`, the code is trying to execute a method named `operator<<` where the arguments are an `ostream` object and a `vector<int>` object, but it cannot find an operator overload with those particular parameter types. This tells us, if we want to make this work, we need to implement a function like:

```
ostream& operator<<(ostream& out, const vector<int>& val);
```

5. (Intermediate) Write code for a template class that can *contain* exactly 5 items of any type, which will be supplied via the constructor. You can store the values internally however you wish, but you must include a getter method that will return a value when supplied the integer index of the value.

E.g., suppose your class is named `holds5`. Then code like the following should work:

```
void holds5_test()
{
    holds5<string> holder("zero", "one", "two", "apple", "four");
    holder.get_item(3);    // returns "apple"
}
```

```
template <typename T>
class holds5 {
public:
    // constructor
    holds5(T a, T b, T c, T d, T e) {
        _data = {a, b, c, d, e}; // this assignment works with vectors, not arrays
    }

    T get_item(size_t index) { return _data.at(index); }
private:
    vector<T> _data;
};
```